1.1:

Supercomputers: Very powerful computer systems that can perform the highest currently possible amount of operations per second.
Personal Computers: Computers found in the everyday home that allow errors to not be catastrophically bad that causes widespread damage. Also allows people to have an accessible computer in their everyday lives.
Server computers: Connect user systems/computers to digital servers, allowing a user to use something or access data through the cloud.
Analog Computers: Uses physical signals to model and affect an algorithm that is solving a problem. Examples include temperature, electricity, etc.

1.3:

- Compiler translates the high level language statements into assembly language statements. Assembly language can be described as being a mix between the computer and brain, as it allows for programmers to command the computer without completely going into binary. The middle-man between a computer and human, as it allows for assembly to be able to read the commands.
- Assembler translates symbolic version of instructions (assembly language) into binary language. Binary language is represented by 1s and 0s and are the direct machine instructions.
- Computer can comprehend and run the binary instructions directly.

1.4:

A:
8 bits that display three colors each = 8 * 3 = 24 bits for color
1280 x 1024 pixels, so byte size = pixels * bytes per pixel.
1280 * 1024 * 3 = **3932160 bytes is the minimum frame buffer size.**
B:
Convert bytes to bits: 3932160 * 8 = 31457280 bits
Divide bits by the network download speed.
31457280/100 * 10^6 b/s = 0.3146 → **.315 seconds**

1.5:

A:
Instructions Per Second = Clock rate / CPI
P1: 3 Ghz / 1.5 = 2 billion instruction per second
P2: 2.5 Ghz / 1.0 = 2.5 billion instructions per second
P3: 4 GHz / 2.2 = 1.82 billion instructions per second
Therefore, P2 has the highest performance with 2.5 billions instructions per second.

B:
Cycles = execution time (10) * clock rate
P1: 3 GHz * 10 sec = 3 * 10^10 cycles
P2: 2.5 GHz * 10 sec = 2.5 * 10^10 cycles
P3: 4 GHz * 10 sec = 4 * 10^10 cycles
Instructions = Cycles / CP1
P1: 3 * 10^10 / 1.5 = **2 * 10^10 instructions**
P2: 2.5 * 10^10 / 1 = **2.5 * 10^10 instructions**
P3: 4.0 * 10^10 / 2.2 = **1.82 * 10^10 instructions**
C:
CPI * 1.2 and Execution time * 0.7 → 7 seconds
Execution time = CPI * Instructions # / Clock Rate → Clock Rate = CPI * Instructions / Exec
P1:
CPI: 1.5 * 1.2 = 1.8
Clock Rate = (1.8 * 2 * 10^10) / 7 → **5.14 Ghz**
P2:
CPI: 1.2 * 1 = 1.2
Clock Rate = (1.2 * 2.5 * 10^10) / 7 → **4.29 Ghz**
P3:
CPI: 1.2 * 2.2 = 2.64
Clock Rate = (2.64 * 1.18 * 10^10) / 7 → **6.85 Ghz**

1.7:
A:
Clock Rate = 10^9 cycles/sec, as 1 sec / 1 ns = 1 GHz = 10^9 cycles per second
CPI = Clock Rate * Execution / Number of instructions
Compiler A: Dynamic Instruction = 1.0 * 10^9 and Exec time = 1.1s
(10^9 cycles/second * 1.1seconds)) / (1.0 * 10^9) = 1.1 / 1,0 = 1.1
CPI = 1.1 cycles
Compiler B: Dynamic Instruction = 1.2 * 10^9 and Exec time = 1.5s
(10^9 cycles/second * 1.5seconds)) / (1.2 * 10^9) = 1.5 / 1.2 = 1.25
CPI = 1.25 cycles
B:
Exec Time = (Instruction # * CPI) / Clock Rate

- Exec time is the same for both compilers, so set the equations equal to eachother.

Compiler A (Instruction # * CPI) / Clock Rate = Compiler B (Instruction # * CPI) / Clock Rate

- Plug in all of previously found values into the calculated equation.

1.0 * 10^9 * 1.1 / (COMP A CR) = 1.2 * 10^9 * 1.25 / (COMP B CR)
(COMP B CR)/(COMP A CR) = (1.1 * 10^9) / (1.2*1.25*10^9) = 1.1/1.5 = 0.733
Compiler A's clock rate is 73.3% as fast as Compiler B's clock rate and vice versa.

C:
Speedup = Old exec time / New exec time
Exec Time = CPI * Instruction Count / Clock Rate
Speedup = ((OLD)CPI * Instruction Count /~~ Clock Rate~~) / ((NEW)CPI * Instruction Count /~~ Clock Rate~~)
New Compiler: Instruction Count = 6.0 * 10^8 and CPI = 1.1
Compiler B:

- Instruction Count = 1.2 * 10^9 and CPI = 1.25

Speedup = (1.2 * 10^9 * 1.25) / (6.0 * 10^8 * 1.1) = 2.273
Therefore, new compiler is 2.27 times faster compared to compiler B
Compiler A:

- Instruction Count = 1.0 * 10^9 and CPI = 1.1

Speedup = (1.0 * 10^9 * 1.1) / (6.0 * 10^8 * 1.1) = 1.667
Therefore, new compiler is 1.67 times faster compared to compiler A.

1.10:

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{yield}}$$

$$\text{Dies per wafer} \approx \frac{\text{Wafer area}}{\text{Die area}}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}/2))^2}$$

A:
Wafer 1 (15)
Area = pi * (15/2)^2 = 176.71 cm^2
Die area = 176.71/84 = 2.10 cm^2
Yield = 1/(1+(0.020 * 1.05)^2 = 1/(1+0.000441) = **.9996**
Wafer 2 (20)
Area = pi * (20/2)^2 = 314.16 cm^2
Die area = 314.16/100 = 3.14 cm^2
Yield = 1/(1+(0.031 * 1.57)^2 = 1/(1+0.002374) = **.9976**
B:
Wafer 1 (15)
Cost per die = $12 / (84 dies * 0.9996 yield) = **$0.143 per die**
Wafer 2 (20)
Cost per die = $15 / (100 dies * 0.9976 yield) = **$0.150 per die**
C:
Wafer 1 (15)
New die amount = 84 * 1.1 = 92.4 or just 92 dies
New die area = 176.71/92 = **1.92 cm^2**
New defects = 0.020 * 1.15 = 0.023 defects/cm
Yield = 1/(1 + (.023 * 0.96)^2 = **.9995**

Wafer 2 (20)
New die amount = 100 * 1.1 = 110 dies
New die area = 314.16/110 = **2.86 cm^2**
New defects = 0.031 * 1.15 = 0.03565 defects/cm
Yield = 1/(1 + (.03565 * 1.43)^2 = **.9974**
D:
Flip Equation → 1 + (DPA * Die Area/2)^2 = 1/Yield → (DPA * Die Area/2)^2 = 1/Yield - 1
Isolate DPA → DPA * Die Area/2 = root(1/Yield - 1)
→ DPA = 2/Die Area * root(1/Yield - 1)
200 mm = 2 cm
Initial Yield:
D = 2/2 * root(1/0.92 - 1) = root(0.08696) = 0.295
Die Area = 0.295 cm^2
Final Yield:
D = 2/2 * root(1/0.95 - 1) = root(0.05263) = 0.229
Die Area = 0.295 cm^2

1.11:

1.11 The results of the SPEC CPU2006 bzip2 benchmark running on an AMD Barcelona has an instruction count of 2.389E12, an execution time of 750 s, and a reference time of 9650 s.

A:
0.333 ns = 0.333 * 10^-9 s → Clock Rate = 1/(0.333 * 10^-9) = 3 * 10^9 cps
CPI = (3* 10^9 cps * 750 seconds) / (2.389 * 10^12 instructs) = 0.9418
CPI = 0.9418
B:
SPEC = Ref Time / Exec Time ?????
9650 sec / 750 sec = 12.87
SpecRatio = 12.87
C:
New Instruction Count: 1.1 * 2.389 * 10^12 = 2.6279 * 10^12
CPU time = (Instruction Count * CPI) / Clock Rate
Since CPU time is proportionate to instruction count, it will also increase by 10%.
New Ref Time: 750 sec * 1.10 = 825 secs
Therefore, CPU time increases by 75 seconds.
D:
Instruction Count: 1.1 * 2.389 * 10^12 = 2.6279 * 10^12
CPI: 0.942 * 1.05 = .9891

- Clock Rate doesn't matter because it doesn't change.

Post CPU time = 750 * 1.10 * 1.05 = 866.25 seconds.

CPU time increases by 116.25 seconds.

E:

9650 seconds / 866.25 seconds = 11.14

Change in SpecRatio = 12.87 - 11.14 = 1.73 lower after increases

F:

Instructions: 2.389 * 10^12 * 0.85 = 2.03065 * 10^12

New Time: 700s

New Clock Rate: 4 * 10^9

CPI = 700 * 4 * 10^9 / 2.03065 * 10^12 = 1.3789

CPI = 1.3789

G:

1.379/0.942 = 1.464 → 46.4 % increase.

4 GHz / 3 GHz = 1.333 → 33.3 % increase

The two increases are not similar as there is a 13 percent difference between the two values. This can be explained by some factors like instructions being different regarding complexity/depth, pipelining, and other things.

H:

750 seconds - 700 seconds = 50 seconds

CPU time has decreased by 50 seconds or about 6.6 percent.

I:

864 ns = 864 * 10^-9 sec

Instructions = Exec Time * Clock Rate / CPI

Instructions = (864 * 10^-9 * 4 * 10^9)/1.61 = 2147.52 instructions

2147.52 or 2147 instructions

I ran out of time. Dock me for these :(

J:

K: